Software Development
and
Professional Practice

Process Life Cycle Models

By

Dr. Rasha Mahmoud Kamel

Agile Development Methodologies

- As opposed to the heavyweight plan-driven models, the new agile process model is lightweight, requiring ***less documentation*** and fewer process controls.
- It is designed for ***small to medium-sized software projects*** and ***small development teams***. It enables these teams to quickly ***adapt to changing requirements and customer demands***.
- It is proposed to release ***completed software much more quickly*** than plan-driven models.
- Agile development is based on the idea that the primary goal of software projects is to deliver working software. Since the focus is on delivering functional code, the development team should spend most of their time writing code, ***not writing documents***.

Agile Development Methodologies

- Lightweight methodologies have several key characteristics. They emphasize ***writing tests before code***, ***frequent product releases***, ***significant customer involvement***, ***common code ownership*** (allowing any developer to modify any part of the code when necessary), and ***refactoring*** (rewriting code to make it simpler and easier to maintain).
- Lightweight methodologies also suffer from several myths. Two of the most persistent are that lightweight processes are ***good only for very small projects*** and that they require no ***process discipline***. "In reality, lightweight methodologies have been successfully applied in many small and medium-sized projects." They also require process discipline, especially in the beginning of a project when initial requirements are defined and an iteration cycle is planned.
- Examples of Agile methodologies include Extreme Programming, Scrum, and Kanban.

eXtreme Programming (XP)

- XP is a lightweight, efficient, low-risk, flexible, predictable, scientific, and fun way to develop software. XP relies on the following four fundamental ideas:

  (1) ***Heavy customer involvement:*** XP requires that a ***customer representative be part of the development team*** and be on site at all times. The customer representative works with the team to create the contents of each iteration of the product and creates all the acceptance tests for each preliminary release.

  (2) ***Continuous unit testing*** (also known as ***test-driven development*** [TDD]): XP ***calls for developers to write the unit tests for any new features*** before any of the code is written. It gives a developer a clear metric for success. When all the unit tests pass, you have finished implementing the feature.

eXtreme Programming (XP)

(3) ***Pair programming:*** XP requires that ***pairs of developers*** write all code–a driver and a navigator–who share a single computer. The driver is actually writing the code while the navigator watches, catching typos, making suggestions, thinking about code structure, and etc. The pair switches places periodically (every 30 minutes or so, or when one of them thinks he has a better way of implementing a piece of code). While a pair of programmers is not quite as productive as two individual programmers when it comes to number of lines of code written per unit of time, their code ***contains fewer defects***. This makes them more productive overall. Pair programming provides the team ***an opportunity to refactor existing code***–to ***redesign it to make it as simple as possible*** while still meeting the customer's requirements. XP was the first discipline to use pair programming, but it can also be used in other Agile methods.

eXtreme Programming (XP)

(4) ***Short iteration cycles and frequent releases:*** XP ***uses release cycles in the range of just a few months*** and each release is divided into smaller iterations (about 4–6 weeks). The combination of frequent releases and an on-site customer representative allows the team to get ***immediate feedback on new features*** and to ***uncover design and requirements issues early***. XP also requires ***constant integration and building of the product***. Whenever developers finish a feature and ***it passes all unit tests***, they immediately integrate it and build the entire product. They then run all unit tests to ***ensure the new feature hasn't broken existing functionality***. If it breaks something, they fix it immediately. So, ***integrations and builds may occur several times a day***. This allows the team to know the status of the release cycle every day and gives the customer a completed build on which to run the acceptance tests.

eXtreme Programming (XP)

- Risk is the most basic problem in software.
- Risk manifests in many ways: ***schedule delays***, ***project cancelation***, ***increased defect rates***, ***misunderstanding of the business problem***, ***false feature rich*** (adding features the customer really doesn't want or need), and ***staff turnover***.
- Managing risk is a very difficult and time-consuming management problem.
- XP seeks to ***minimize risk by controlling the four variables*** of software development: cost, time, scope, and quality. XP recognizes that to minimize risk, developers need to control as many of the variables as possible, but especially they need to control the scope of the project.

XP – The Four Basic Activities

- XP describes four activities that are the foundation of the discipline:

  (1) Coding: Writing the actual code is at the heart of XP (it is your main activity). The fundamental difference between plan-driven models and agile models is this emphasis on code. In agile models, ***the code is the sole deliverable,*** so the focus is placed completely on it. In addition, ***by structuring the code properly and keeping comments up to date***, the code becomes documentation for the project.

  (2) Testing: In XP, unit tests play a critical role in the development process. Developers ***write unit tests before writing the actual code*** that they test. The tests ensure that the code meets the expected behavior—when all tests pass successfully, the feature is considered complete. XP depends on using an automated testing framework ***to run all the unit tests whenever changes are integrated***.

XP – The Four Basic Activities

(3) Listening: To your partner and to the customer. In any given software development project ***there are two types of knowledge: domain knowledge*** and ***technical knowledge***. ***Domain knowledge*** of the project refers to the knowledge of the business application being written and its intended functionality (what it is supposed to do). It involves knowledge of the specific industry, business, or application area where the software will be used. This knowledge is typically possessed by the customer. ***Technical knowledge*** refers to the expertise in the target platform, the programming languages, tools, and technologies required to build the software. This knowledge is typically possessed by the developers. The customer doesn't know the technical side and the developers don't have the domain knowledge, so listening is a key activity in developing the product.

XP – The Four Basic Activities

(4) Designing: Designing is creating a structure that organizes the logic in the system. **Good design** ***organizes the logic*** so that a change in one part of the system doesn't always require a change in another part of the system. **Good design** ***ensures that every piece of logic in the system has one and only one home.*** **Good design** ***allows the extension of the system with changes in only one place.***

XP – The Four Values

- In XP, there are the following four core values that enable it to work:

  (1) Communication: This refers to ***sharing the team's collective knowledge among all members***. Keeping the team small facilitates communication. Pair programming and collective ownership also facilitate communication by spreading the knowledge of the codebase around the entire team. XP developers are encouraged to fix bugs they find and redesign features to improve simplicity.

  (2) Simplicity: XP focuses on developing ***the simplest piece of software*** that solves today's task. XP developers bet that "it is better to do a simple thing today and pay a little more tomorrow to change it if it needs it, than to do a more complicated thing today that may never be used anyway". All developers on an XP team are allowed and encouraged to ***redesign code to make it simpler*** at any time (refactoring).

XP – The Four Values

(3) Feedback: XP programmers are required to ***write tests before they write the code***, so that they always have ***immediate feedback about their code*** and its impact on the system. Also, the customer is writing functional (acceptance) tests so those are available to measure how well the system is adhering to the "stories" used to develop it.

(4) Courage: XP developers ***must have courage***. They must be willing to make changes at any time when the design no longer fits. They need to be prepared to throw code away if it doesn't work.

XP – The 15 Principles

- From the four values, XP derives some basic principles. The principles are as follows:

  (1) Rapid feedback: ***Get feedback***, understand it, and ***incorporate it into the system as quickly as possible***.

  (2) Assume simplicity: Focus on today's task and ***solve it in the simplest way possible***. Simplify the code whenever you are making changes (refactoring) to keep it ***as simple as possible and reduce defects***.

  (3) Incremental change: ***Integrate your new code into the system every day or whenever you finish a task***. This allows you to find interface and interaction errors quickly and gives the customer a new baseline to examine at least once a day.

  (4) Quality work: Strive for ***defect-free code***. Pair programming and test-driven development help reduce defects in the code.

XP – The 15 Principles

(5) Embrace change: Change is inevitable in software development, so be prepared for it. ***Agile methodologies*** such as XP are built on the idea ***that change is constant.*** The more a development process accommodates change, the more effective it becomes.

(6) Teach learning: XP teams are encouraged to develop the ability to ***continuously learn and improve*** rather than strictly follow predefined rules. The goal is to understand the purpose behind practices such as testing, refactoring, and coding standards. For example, instead of saying "You must write unit tests exactly like this template," XP encourages understanding why testing matters, how tests improve design, and how to write effective tests.

XP – The 15 Principles

(7) Small initial investment: Beginning a project with ***limited resources and a constrained budget*** encourages teams to focus on ***simple design*** and implementation. This reinforces the simplicity by discouraging unnecessary complexity.

(8) Work with people's instincts, not against them: People like winning, interacting with others, being part of a team, seeing their code work, and being trusted. Don't do things that go against this. By providing ***the team with the environment and support they need*** and fostering trust, they are more likely to perform effectively and deliver high-quality results.

XP – The 15 Principles

(9) Open, honest communication: Team members must be able to ***criticize constructively*** and ***deliver both positive and negative feedback.*** This principle has two main aspects. First, the goal of all team members is to improve the quality of the code, so constructive criticism contributes to better outcomes. Second, collective code ownership ensures that all team members can modify and improve the codebase without fear of hurting someone else's feelings.

(10) Travel light: It means that the team should avoid unnecessary complexity. ***Keeping only essential artifacts***—things that provide long-term value—***and dispose things*** (code, design) ***that are no longer needed.***

XP – The 15 Principles

(11) Concrete experiments: Making abstract decisions about requirements or design without testing them may result in ***wasted time and effort***. To avoid this, XP encourages concrete experiments—practical ways to evaluate whether a decision is feasible before fully committing to it. One common technique is to ***produce a spike which is a quick and dirty proof-of-concept piece of code*** that implements the outline of the decision so the team can see if they are actually right or not. If they are wrong, they have not wasted lots of time figuring that out.

XP – The 15 Principles

(12) Accepted responsibility: The team as a whole ***is responsible for the success of the product***. Responsibility is accepted by the entire team and tasks are not assigned. XP teams typically ***do not have managers who assign tasks*** or control the workflow. Instead, XP teams follow a self-organizing approach in which team members ***select their own tasks and make sure they get done.*** Common code ownership naturally leads to common project ownership. While XP teams typically do not have managers, they may ***have a coach who guides the process*** and a project manager who handles the administrative tasks and responsibilities.

XP – The 15 Principles

(13) Play to win: If the team members are not ***worried about schedules or constantly changing requirements,*** they will be able to ***focus on the work at hand*** rather than impending deadlines. This focus enables the team to generate cleaner, higher-quality code and increases their productivity. Maintaining a calm and focused mindset contributes to overall project success.

(14) Local adaptation: ***XP is flexible and can be adapted to local needs and circumstances of a project.*** The development team collectively owns both the project and the process, and any adaptations to XP practices are agreed upon through team consensus.

XP – The 15 Principles

(15) Honest measurement: This principle emphasizes ***measuring the actual progress of a project*** using clear, transparent, and meaningful metrics, such as working software, customer satisfaction, team performance and productivity, defects, and software quality. Accurate measurements are necessary to guide the project and evaluate its progress, as well as help the team stay on track, make informed decisions, and continuously improve the development approach.

XP – The 12 Practices

- Implementing XP may vary depending on the team and the project, but should involve some form of the following practices:

  (1) Small releases:

  - Put a simple system into production quickly, and then ***release new versions on a very short cycle***.
  - It is better to plan releases in ***durations of* 1–2 *months*** rather than 6–12 months.
  - Each release has to ***make sense from a business perspective***, so release size will vary.
  - The larger the release scope and duration, the harder it is to estimate.

XP – The 12 Practices

(2) On-site customer:

- ***A customer is part of the team***, is on-site, writes and executes functional tests, and helps clarify requirements.
- The customer's ability to give immediate feedback to changes in the system also increases team confidence that they are building the right system every day.

(3) Collective ownership:

- The team owns the entire project and all of its code, implying that ***anyone can change anything at any time***.
- Developers need to buy into the idea that anyone can change their code and that collective ownership extends from code to the entire project.

XP – The 12 Practices

(4) Simple design:

- ***Keep the design as simple as you can*** each day. Redesign often to keep it simple.
- A simple design should **(1)** pass all the unit tests, **(2)** have no duplicated code, **(3)** express what each story means in the code, and **(4)** have the fewest number of classes and methods that make sense to implement the stories so far.

(5) Testing:

- ***Programmers*** constantly ***write unit tests and customers*** constantly ***write functional tests.***
- Tests must all pass before integration. Any program feature without an automated test simply doesn't exist.

XP – The 12 Practices

(6) Refactoring:

- ***Restructure the system to make it simpler without changing its behavior*** by removing redundancy, eliminating unnecessary code, or adding flexibility.
- The key to refactoring is to ***identify areas of code that can be made simpler and to do it.*** This makes the code cleaner, less redundant, more readable, more efficient, and more maintainable.
- Refactoring is closely related to collective ownership and simple design.
- Collective ownership gives you permission to change the code written by others and simple design imposes on you the responsibility to make the change when you see it needs to be made.

XP – The 12 Practices

(7) Pair programming:

- ***All production code*** must be written ***by two programmers at one machine***.
- Pair programming is a dynamic process. You may change partners as often as you change tasks to implement.
- This has the effect of ***reinforcing collective ownership*** by spreading the knowledge of the entire system around the entire team.
- It avoids the "beer truck problem", where the person who knows everything gets hit by a beer truck and thus sets the project schedule back months.

XP – The 12 Practices

(8) Coding standards:

- Because of ***collective code ownership*** and ongoing refactoring, the team must have ***coding standards and everyone must adhere to them***.
- Without a sensible set of coding guidelines, it ***would take much, much longer to do refactoring*** and it would ***decrease the desire of developers to change code***.
- Coding standards ***make the code easier to read and maintain***.

(9) Continuous integration:

- ***Integrate and build the system every time a task is finished***, which may occur several times a day. This helps to isolate problems in the codebase, because if you integrate a single task and an issue arises, then it is most likely caused by the most recent change.

XP – The 12 Practices

(10) 40-hour week:

- ***Work a regular 40-hour week.*** The XP philosophy is that ***the people are less productive if they are working more than 40 hours a week*** than if they are working 40 hours.
- ***Constantly being under deadline pressure*** and never getting a break also means you ***get tired and then make more mistakes***, which somebody then needs to fix.
- ***But working 40-hours a week leaves you with time for a life***, time to relax and recharge, and time to focus on your work during the workday, making you more productive, not less.

XP – The 12 Practices

(11) Planning game:

- ***Develop the scope of the next release*** by combining business priorities and technical estimates. The customer and the development team need to ***decide on the stories/features that will be included in the next release***, ***the priority of each story***, and ***when the release needs to be done***. The developers are responsible for ***breaking these stories into a set of tasks and estimating the duration of each task***. The sum of the durations will determine which tasks can get done before the release delivery date. If necessary, stories are moved out of a release if the numbers don't add up. Notice that ***estimation is the responsibility of the developers*** and not the customer or the manager. In XP only the developers do estimation.

XP – The 12 Practices

(12) Metaphor:

- A simple shared story of how the whole system works.
- The metaphor needs to be ***a coherent explanation of the system*** that is decomposable into smaller bits called stories. ***Stories should always be expressed in the vocabulary of the metaphor*** and the language of the metaphor should be common to both the customer and the developers.
- ***The metaphor is a naming concept for classes and methods*** that should make it easy for a team member to guess the functionality of a particular class/method, from its name only.

Thank you